From Linear Scores to a Single Hidden Layer

A Comparative Study of Learning Systems

Fidan Allahverdiyeva, Ughur Mammadov, Samra Abdullayeva, Yusif Yusifli

UFAZ | AI project

April 17, 2026

Softmax Regression: Mathematical Foundation

The Linear Mapping:

$$\mathbf{s} = \mathbf{w}\mathbf{x} + \mathbf{b}$$

Where $\mathbf{x} \in \mathbb{R}^d$, $\mathbf{w} \in \mathbb{R}^{k \times d}$. **The Probabilistic Map:**

$$P(y = j|\mathbf{x}) = \frac{\exp(s_j)}{\sum_{\ell=1}^{k} \exp(s_\ell)}$$

The Objective Function

We minimize the **Cross-Entropy Loss**:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^{n} \sum_{j=1}^{k} Y_{ij} \log(P_{ij})$$

This is the negative log-likelihood of the Bernoulli/Multinomial distribution.

Implementation Logic: The Log-Sum-Exp Trick

Direct exponentiation of large scores $\mathbf{s}$ leads to numerical `inf` (overflow).

Stability Formula

For each sample, we shift scores by their maximum value $M = \max(\mathbf{s})$:

$$p_j = \frac{\exp(s_j - M)}{\sum_\ell \exp(s_\ell - M)}$$

Gradient of Loss w.r.t Scores:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{s}} = \mathbf{P} - \mathbf{Y}$$

This elegant result allows for efficient backpropagation in the linear case.

Softmax Performance on Synthetic Data

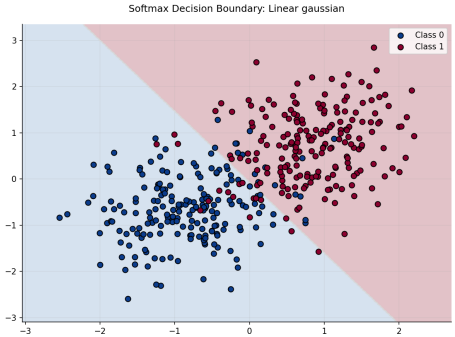


Figure: Gaussian Blobs (Linear)

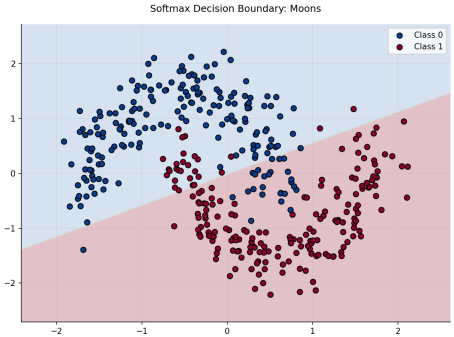


Figure: Moons (Non-Linear)

- **Blobs:** Softmax finds the optimal hyperplane. Accuracy: 99%+.
- **Moons:** The linear constraint causes a "underfitting" failure. Accuracy: 85%.

Digits Benchmark: Training Dynamics

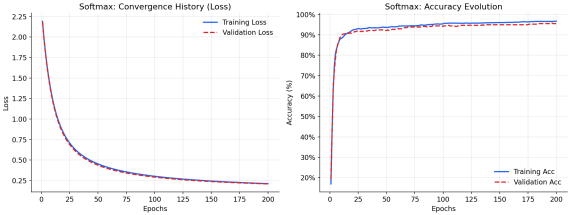


Figure: Convergence History (Loss)

- **Stability:** Training and validation curves track closely.
- **Convergence:** Smooth monotone decrease confirms high-quality learning rate ($\eta = 0.05$).
- **Final Checkpoint:** Best-validation-loss checkpoint restored for evaluation.

Neural Network Architecture: The Hidden Layer

To break the linear constraint, we introduce a hidden representation $\mathbf{h} \in \mathbb{R}^m$:

1. **Hidden Projection:** $\mathbf{z_1} = \mathbf{w}_1\mathbf{x} + \mathbf{b}_1$
2. **Non-linear Activation:** $\mathbf{h} = \tanh(\mathbf{z_1})$
3. **Output Projection:** $\mathbf{s} = \mathbf{w}_2\mathbf{h} + \mathbf{b}_2$
4. **Softmax:** $\mathbf{P} = \text{Softmax}(\mathbf{s})$

Universal Approximation Theorem

A single hidden layer with a non-linear activation can approximate any continuous function, provided the width m is sufficient.

Optimization: Manual Gradient Descent

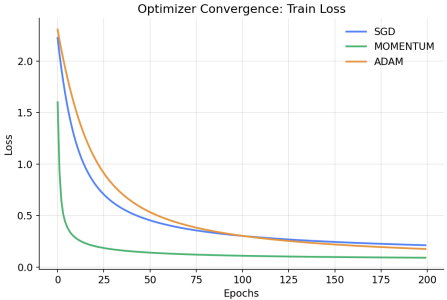
We implemented the chain rule manually for the NumPy engine:

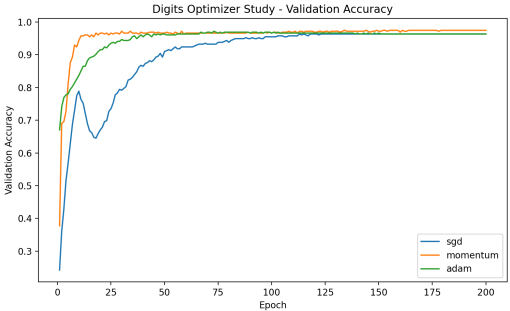
- **Output Layer:** $\delta_2 = \mathbf{P} - \mathbf{Y}$
- **Weight 2 Gradients:** $\nabla \mathbf{w}_2 = \delta_2^T \mathbf{h}$
- **Hidden Layer Error:** $\delta_1 = (\delta_2 \mathbf{w}_2) \odot (1 - \tanh^2(\mathbf{z_1}))$
- **Weight 1 Gradients:** $\nabla \mathbf{w}_1 = \delta_1^T \mathbf{x}$

The Update Rule: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \mathbf{w}$

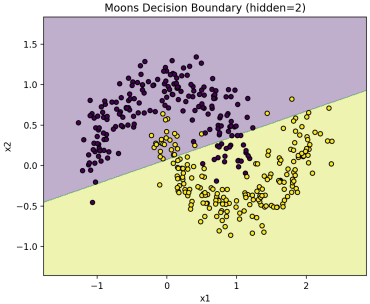
Comparing Optimizers: SGD, Momentum, and Adam

- **SGD:** Standard gradient descent. Often slow and sensitive to learning rate.
- **Momentum:** $v_t = \gamma v_{t-1} + \eta \nabla \mathcal{L}$. Helps dampen oscillations.
- **Adam:** Adaptive Moment Estimation. Maintains moving averages of gradients (m_t) and squared gradients (v_t).

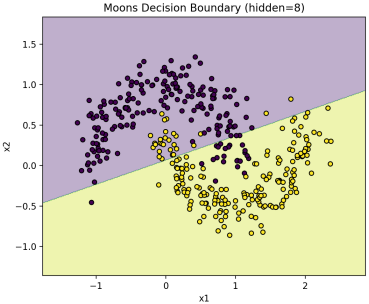




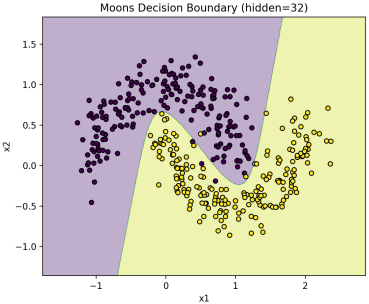
NN Experimental Results: Solving Non-Linearity



$m = 2$ (Linear-like)



$m = 8$ (Struggling)

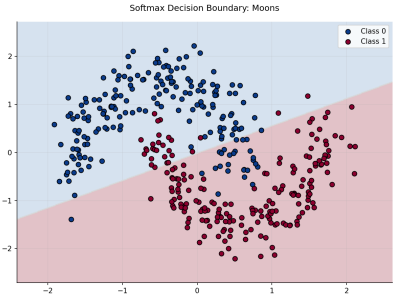


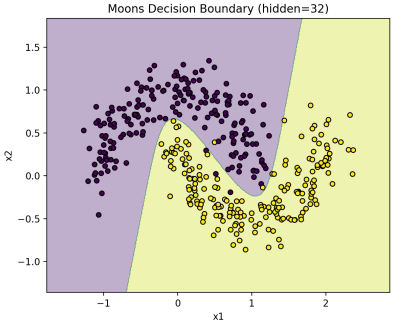
$m = 32$ (Optimal)

- **Result:** Width m acts as a capacity threshold.
- The NN achieves **97.5% Acc** on Moons, outperforming Softmax by 12.5%.

Direct Comparison: Linear vs. Hidden Layer

Dataset	Softmax Acc	NN Acc	Winner	Reasoning
Gaussian Blobs	99.2%	99.2%	**Softmax**	Lower Complexity
Moons	85.0%	97.5%	**NN**	Geometric Curvature
Digits (MNIST)	93.5%	95.4%	**NN**	Feature Extraction





Summary of Findings

Task Type	Winner	Why?
Linear Gaussian	Softmax Regression	Equal performance, simpler model.
Non-linear Moons	NN ($m = 32$)	Structural necessity for curved boundaries.
Large Benchmark	NN (Adam)	Statistical Acc boost and better calibration.
Low-Width Moons	Softmax/Tie	Hidden layers need width to be effective.

Final Verdict

Representational complexity is only justified when task geometry demands it.

Measuring Model Confidence

A model should not only be accurate but also **calibrated**.

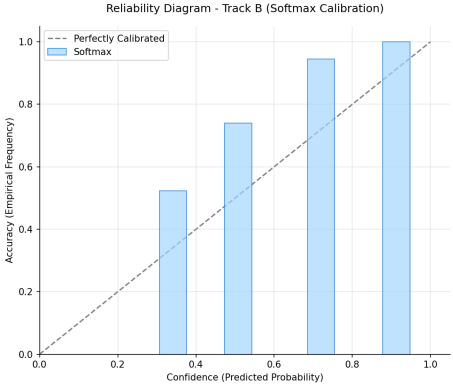
Predictive Entropy (H)

$$H(\mathbf{x}) = -\sum_{j=1}^{k} p_j \log p_j$$

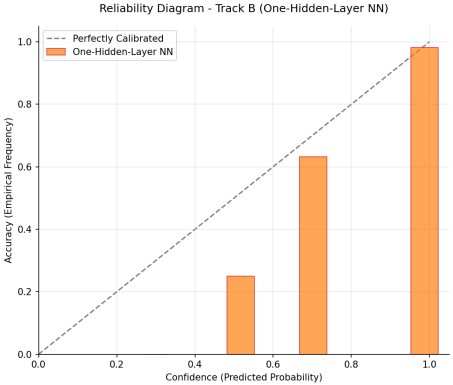
H is high when the model is uncertain (e.g., $p = [0.5, 0.5]$) and low when certain.

Reliability Analysis: We bin predictions by confidence and check if *Confidence* $\approx$ *Accuracy*.

Reliability Diagram Results



Softmax (Overconfident)



NN (Better Calibrated)

- **Finding:** The NN is more "honest" about its mistakes.
- **Softmax Error Confidence:** 60.2%
- **NN Error Confidence:** 54.1% (Closer to random 50% on binary tasks)

Conclusion

- **Softmax** is superior for simple, high-dimensional linear tasks due to interpretability and speed.
- **Neural Networks** are essential when the decision boundary requires "warping" of the input space.
- **Optimization Matters:** Adam provides the most stable convergence for the NN architecture.
- **Calibration:** Adding a hidden layer improves not just the labels, but the quality of the predicted probabilities.

Limitations

- **Scale:** Digits is a small dataset; results might vary on ImageNet or larger tasks.
- **Depth:** We only looked at $L = 1$ hidden layer. Deeper networks (CNNs) might reveal different trade-offs.
- **Seeds:** While 5 seeds give a 95% CI, more seeds (e.g., 30) would improve Gaussian normality assumptions.

Thank You!

Any Questions?